

CYBERFORGE: BANKING SENTINEL

Complete Technical Demonstration Defense & Evaluator Q&A; Guide

Purpose of this Document: This guide provides granular technical answers, mathematical foundations, and architectural justifications for every potential question an evaluator or professor may ask during the live demonstration of the CyberForge project. It details the exact code locations and explicitly notes what is natively implemented versus prototype simulations.

1. Authentication & Cryptographic Storage

Q1: How are you storing passwords securely in the database?

Exact Code Reference: *File:* `auth.py` | *Function:* `hash_password()`
Uses **PBKDF2-HMAC-SHA256** with **260,000 iterations** and a **256-bit cryptographically secure random salt** (`os.urandom(32)`). Stored format: `iterations$salt_hex$hash_hex`.

Recommended Defense Answer: "We use PBKDF2-HMAC-SHA256 compliant with NIST SP 800-132 and OWASP 2023 recommendations. Instead of fast reversible encryption or vulnerable simple hashes like MD5/plain SHA256, we generate an independent 256-bit salt per user and apply 260,000 hashing rounds. This computationally exhausts attackers, making GPU brute-forcing and Rainbow Table dictionary lookups infeasible."

Q2: What happens during login verification? Do you decrypt the stored password?

Exact Code Reference: *File:* `auth.py` | *Function:* `verify_password()`
Extracts salt & iteration count from stored string, computes PBKDF2 on provided password, and calls `hmac.compare_digest()`.

Recommended Defense Answer: "No, cryptographic hashes are strictly one-way and mathematically non-reversible. During login, our server extracts the user's specific salt from the database, re-derives the key with the supplied password, and compares the outputs using constant-time comparison (`hmac.compare_digest`) to prevent timing-based side-channel attacks."

Q3: How is the session managed after login? Are you using JWT or Cookies?

Exact Code Reference: *File:* `script.js` (lines 75-80)
Current frontend stores `cf_user_id` and `cf_username` in browser `localStorage`.

Recommended Defense Answer: "In our current prototype, session identification is managed via client-side local storage for rapid UI state management. In a full enterprise production rollout, this architecture integrates signed short-lived JSON Web Tokens (JWT) coupled with `HttpOnly`, `SameSite=Strict` cookies to eliminate Cross-Site Scripting (XSS) token theft."

2. Profile & Adaptive Security Configuration

Q4: Why do you maintain both a 4-Digit PIN and a Login Password?

Exact Code Reference: *File:* `models.py` (`transaction_pin` column) & `main.py` (line 125)
PIN is verified only when transaction risk score is < 25 (Low Risk).

Recommended Defense Answer: "We implement Multi-Tier Step-Up Authentication (Risk-Based Authentication). The master password secures account access, while the 4-digit PIN provides a low-friction second factor for familiar, low-risk micro-transactions (analogous to UPI PINs), reserving disruptive OTPs strictly for anomalous behavior."

Q5: How is the registered email protected and validated?

Exact Code Reference: File: `main.py (mask_email()) & script.js`

RFC 5322 regex validation on input; server masks email string before returning to client (e.g. `a***9@gmail.com`).

Recommended Defense Answer: "We enforce regex validation at ingestion. Furthermore, under Privacy-by-Design and DPDP/GDPR principles, all API responses and frontend views mask Personally Identifiable Information (PII) so cleartext emails are never displayed on screens or stored in client logs."

3. Client Telemetry & Behavioral Biometrics

Q6: How do you capture typing speed? Is this a keylogger?

Exact Code Reference: File: `script.js (lines 260-275) | trackKeystroke()`

Measures `typingSpeedMs = Date.now() - firstKeystrokeTime`. Characters are never inspected.

Recommended Defense Answer: "No, this is strictly Privacy-Preserving Behavioral Biometrics. We do NOT capture keystroke characters, which would constitute a severe keylogging vulnerability. We only record the temporal latency (time delta in milliseconds) between initial interaction and submission. This reliably differentiates genuine human typing cadence (1.5s to 8s) from automated bot pastes (< 500ms)."

Q7: How are Client IP and Device fingerprint extracted? Can they be spoofed?

Exact Code Reference: File: `main.py (line 79) | request.client.host`

Server extracts IP directly from the TCP socket connection, ignoring spoofable client headers.

Recommended Defense Answer: "To prevent client payload manipulation, the client IP is extracted directly at the socket level on the backend via `request.client.host`. The device fingerprint utilizes `navigator.userAgent`, and historical familiarity is established by querying past completed transactions for that user in MySQL."

4. AI Risk Engine & Decision Matrix

Q8: What exact Machine Learning model is used and how is risk computed?

Exact Code Reference: Files: `predictor.py & train_model.py`

Model: Scikit-Learn `IsolationForest` (150 estimators) + 5-factor weighted ensemble + deterministic guardrails.

Recommended Defense Answer: "We employ a Hybrid Ensemble Risk Model. It evaluates a 5-dimensional feature vector: Network IP Context (25%), Device Fingerprint (20%), Typing Cadence vs Baseline (15%), Amount Deviation (25%), and Circadian Time Window (15%). An unsupervised Isolation Forest detects multi-dimensional anomaly clustering. Deterministic Banking Guardrails enforce maximum risk floors (≥ 80) if transactions exceed Rs. 50,000 or bot speed is detected."

Q9: Why choose Isolation Forest over Deep Neural Networks (DNN)?

Exact Code Reference: File: `train_model.py`

Recommended Defense Answer: "1) Unsupervised Anomaly Isolation: Real banking fraud is rare (<0.1%) with constantly evolving zero-day attack patterns; supervised DNNs suffer from severe class imbalance. 2) Real-time Latency: Sub-10ms CPU inference time. 3) Green Computing: Zero GPU compute overhead."

Q10: What are the decision score thresholds and associated actions?

Exact Code Reference: File: `main.py (lines 125-165)`

Recommended Defense Answer: "Score < 25 (Low Risk): REQUIRE_PIN (frictionless 4-digit PIN). Score 25 to 79 (Medium Risk): REQUIRE_OTP (dispatches 6-digit Email OTP via SMTP). Score >= 80 (High Risk): REQUIRE_HARDWARE_AUTH (locks transfer into ESP32_Awaiting state)."

5. Out-of-Band OTP Generation & Delivery

Q11: How are OTPs generated and replay attacks prevented?

Exact Code Reference: *Files:* otp_service.py & main.py (/api/otp/verify)
Table: otp_records (fields: otp_code, created_at, is_used).

Recommended Defense Answer: "OTPs are generated on the backend and committed to MySQL. We mitigate replay and brute force attacks through: 1) Single-Use Invalidation: upon successful verification, is_used is immediately flagged True. 2) Time-to-Live: strict 5-minute expiration window. 3) Resend Revocation: generating a new OTP automatically invalidates all previous unused tokens."

Q12: What protocol and encryption are used to send the OTP email?

Exact Code Reference: *File:* otp_service.py | smtplib.SMTP_SSL('smtp.gmail.com', 465)

Recommended Defense Answer: "We communicate via direct SMTP wrapped in end-to-end Transport Layer Security / SSL over Port 465. Authentication uses Google's 16-character App Password stored in backend environment variables, avoiding cleartext credentials in source control."

6. Hardware Security Token (ESP32 Integration)

Q13: How does the hardware authentication work with the ESP32?

Exact Code Reference: *File:* main.py (/api/hardware/pending_requests & /api/hardware/verify)

Recommended Defense Answer: "When risk is >= 80, the backend sets status to 'ESP32_Awaiting'. The ESP32 IoT microcontroller polls the REST API over Wi-Fi. Upon detecting a pending high-risk transaction, it displays the amount on an OLED screen and requires physical push-button confirmation, posting back an APPROVED payload to complete the transfer."

7. Database Architecture & Audit Logging

Q14: What database is used and how is non-repudiation maintained?

Exact Code Reference: *Files:* models.py & database.py | MySQL 8.0 with InnoDB engine

Recommended Defense Answer: "We utilize MySQL 8.0 with SQLAlchemy ORM, ensuring full ACID transactional compliance. Non-repudiation and regulatory auditing are guaranteed via the audit_logs table, which writes immutable records containing transaction IDs, calculated risk scores, active anomaly flags, and authorization reasons with UTC timestamps."

8. Implementation Status Matrix (Native vs Prototype)

Feature Component	Implementation Status	Technical Details & Notes
Password Hashing	100% Native	PBKDF2-HMAC-SHA256, 260k iterations, 256-bit salt in auth.py.

Email OTP Dispatch	100% Native	Real SMTP over SSL (Port 465) via Gmail App Password.
Behavioral Biometrics	100% Native	DOM keystroke latency timers in milliseconds (Privacy-by-Design).
AI Risk Engine	100% Native	Isolation Forest + 5-feature baseline scoring + banking guardrails.
Relational Persistence	100% Native	MySQL 8.0 with SQLAlchemy ORM and audit logging.
Hardware Token (ESP32)	API-Ready	Endpoints active in main.py; can be verified via REST or physical IoT.
Core Banking Core	Simulated	Simulated wallet balance for risk engine demonstration.